

# The Lumon Innie Task Scheduler - Concurrent Execution System



## Overview

Welcome to Lumon Industries. You are building an interpreter for the Innie task execution system. At Lumon, each Innie (work identity) is assigned a series of tasks that must execute concurrently in the severed work environment. Each task can depend on outputs from other tasks, and workers (Outies, controlled by threads) execute these tasks in parallel.

The key challenge is managing concurrent task execution with proper dependency resolution while maintaining the illusion of seamless, atomic work - no Innie should see the work of another until it's completely finished.



## Core Concepts

### Innies



Each Innie is a task with:

- A unique employee code (ID, e.g., "HELLY", "MARK", "IRVING")
- A sequence of work instructions (assembly-like language)
- Internal state (the work they're doing)
- Zero or more **WAFFLE** commands that finalize their work output

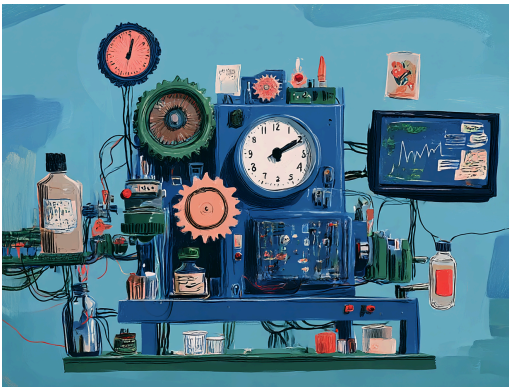
### Waffle Party Output

When an Innie executes a **WAFFLE** command, their work is finalized and exposed to the Outies - this is the output that other Innies can reference and depend on.

*(The name is a reference to Lumon's most insane benefit.)*



### Execution Model



- Multiple Outies (worker threads) manage Innies concurrently
- Each Innie's workday executes **atomically** from start to finish
- If an Innie references another Innie's work output and that output hasn't been finalized yet, the Innie **waits at their desk** until it is
- When an Innie completes all their work (including multiple **WAFFLE** finalizations if present), other waiting Innies receive the **latest** work product



## Work Orders (Commands)

### Basic Operations

- **LOAD** <value> - Load data into your work station
- **ADD** <value> - Add to your current work output
- **MULTIPLY** <value> - Multiply your current work output
- **MODULO** <value> - makes your current work output be the modulo of your current work output % value
- **WAFFLE** - Finalize and submit your work for the day (makes it visible to other Innies)

### Multi-Innie Collaboration

- **ADD** [HELLY, MARK, IRVING] - Wait for all referenced Innies to finalize, then add their work to yours
- **MULTIPLY** [HELLY, MARK] - Similar to ADD but multiplies

### Conditional Work Orders

All conditional operations use the same condition syntax:

#### WELLNESS\_CHECK

**WELLNESS\_CHECK** <condition>

If the condition is true, void your work output (reset to 0). This represents the dreaded wellness event interrupting your work.

#### CONDITIONAL\_ADD

**CONDITIONAL\_ADD** [list] IF <condition>

If the condition is true, wait for all Innies in [list] to finalize and add their work. If the condition is false, skip the ADD. Strategic work prioritization.

### Conditions

Conditions determine when to unblock and evaluate:

- **Simple comparison:** **HELLY > 5 , MARK <= 10**
- **ANY OF:** **HELLY > ANY OF [MARK, IRVING]** - Unblock and evaluate as soon as ONE Innies condition is met. Short-circuits on the first match.



- **ALL OF:** `HELLY > ALL OF [MARK, IRVING]` - Unblock and evaluate when ALL Innies' conditions are met, or short-circuit if one makes the statement false.

## Work Shifts (Loops)

```
SHIFT <number> TIMES
  <work instructions>
END_SHIFT
```

Repeat work instructions atomically. Multiple **WAFFLE** commands within a shift produce multiple work products; waiting Innies receive only the final product.

## Deadlock Detection



Innies may create a circular dependency, in order to avoid this inefficiency and not to waste time, when detected a circular dependency between innies, they will **WAFFLE** to the value of **-1**.

Please keep in mind that work schedules are always deterministic, even if there is a deadlock.

## Work Schedule Format

```
{
  "innies": [
    {
      "id": "HELLY",
      "schedule": "LOAD 10\nWAFFLE"
    },
  ],
}
```

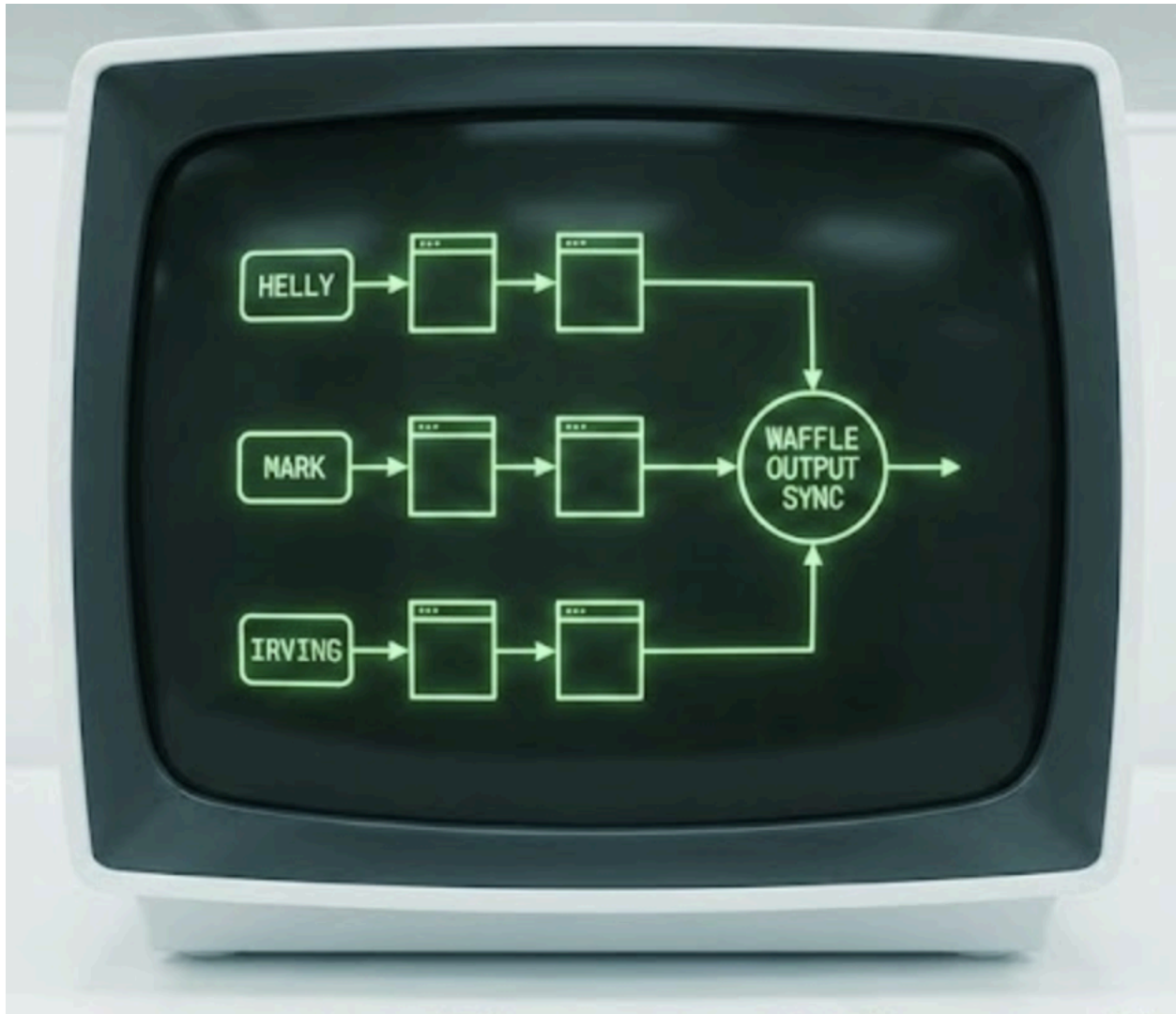


```
{
  "id": "MARK",
  "schedule": "LOAD 5\nADD HELLY\nWAFFLE"
},
{
  "id": "IRVING",
  "schedule": "LOAD 0\nCONDITIONAL_ADD [HELLY, MARK] IF HELLY > ANY OF
[MARK]\nWAFFLE"
}
]
```

## Key Requirements

1. **Concurrent Innie Execution:** Multiple Innies must work simultaneously without conflicts or data corruption
2. **Atomic Workday:** Each Innie's schedule runs atomically from arrival to departure; no partial work products are visible
3. **Dependency Resolution:** Innies automatically wait for dependencies but only when needed
4. **Short-Circuit Evaluation:** ANY OF and ALL OF should evaluate efficiently, unblocking as soon as the result is determined
5. **Latest Product Semantics:** If an Innie WAFFLEs multiple times, waiting Innies receive only the final product
6. **Deterministic Results:** Despite concurrent execution, results should be reproducible and consistent (the severed mind works the same way every time)

## Example Workday



```

{
  "innies": [
    {
      "id": "HELLY",
      "schedule": "LOAD 10\nWAFFLE"
    },
    {
      "id": "MARK",
      "schedule": "LOAD 2\nWAFFLE"
    },
    {
      "id": "IRVING",
      "schedule": "LOAD 0\nADD [HELLY, MARK]\nWAFFLE"
    }
  ]
}
  
```



```
},  
{  
  "id": "BURT",  
  "schedule": "LOAD 100\nWELLNESS_CHECK IRVING > ALL OF [HELLY,  
MARK]\nWAFFLE"  
}  
]  
}
```

Execution flow:

1. **HELLY** and **MARK** can start immediately (no dependencies)
2. **IRVING** waits until both **HELLY** and **MARK** have WAFFLEd, then adds  $10 + 2 = 12$
3. **BURT** waits until **IRVING** WAFFLEs (value: 12), then evaluates if  $12 > 10$  AND  $12 > 2$  (true), so a wellness event resets BURT's output to 0
4. Everyone clocks out, having completed their work orderly and without incident

## Operational Challenges

- Detecting and preventing circular work dependencies (an infinite workday nightmare)
- Managing thread safety for shared work product state
- Efficiently determining when an Innie can resume work
- Handling ALL OF short-circuit logic correctly
- Supporting nested work shifts with dynamic dependencies
- Ensuring no deadlocks or resource starvation (Innies must eventually clock out)

## Success Criteria

Your Lumon work scheduler should:

- Parse the work schedule JSON correctly
- Execute Innie workdays concurrently using multiple Outies (threads/workers)
- Handle all work order types correctly
- Resolve dependencies without race conditions or data corruption
- Produce correct, deterministic work products
- Handle edge cases (circular dependencies, invalid references, etc.)
- Ensure all Innies eventually clock out without hanging



## Resources

Associated with the hometask file, you received several json files with various cases to help you test the scheduler.

- 1.json - desired output is -

|               |    |
|---------------|----|
| <b>HELLY</b>  | 20 |
| <b>MARK</b>   | 5  |
| <b>IRVING</b> | 25 |
| <b>BURT</b>   | 0  |

- 2.json - desired output is -

|              |     |
|--------------|-----|
| <b>DYLAN</b> | 100 |
| <b>BURT</b>  | 200 |

- 3.json - desired output is -

|               |     |
|---------------|-----|
| <b>HELLY</b>  | 8   |
| <b>MARK</b>   | 16  |
| <b>IRVING</b> | 124 |
| <b>BURT</b>   | 497 |
| <b>DYLAN</b>  | 0   |

## General Guidelines

- Please invest up to 4 hours in this task; there's no need to provide a complete solution.

## In our next meeting

- We would like you to walk us through your thought process and discuss possible architectures for handling such a task on a large scale.
- We would like to discuss ideas you did not pursue due to time constraints.

## Outputs

- Solution source code (could be a partial solution)
- Readme.md explaining your code, thought process and future things you would have liked to do (given more time...)
- A List of reading materials you used



Welcome to Lumon Industries. Your cooperation is appreciated.